Testing Documentation

Insighters Inc. Workflow Automation

By: Ayush Singh

Table of Contents

1. Components / Services to Test 3
2. Significant Test Cases
   i) Authentication 5
   ii) Intake Form Submission 7
   iii) Database Storage 9
   iv) PDF Generation 10
   v) Monday.com Integration 11
   vi) Confirmation Email 12
   vii) Docker Startup 13
3. Test Scripts
   i) Setup and Running Tests 14
   ii) conftest.py — Shared Fixtures and Helpers 15
   iii)) Authentication Tests 16
   iv) Form Submission Tests 17
   v) Database Tests 18
   vi) Monday.com Tests 19
   vii) Docker Tests 20

1. Components / Services to Test

The Insighters Inc. Workflow Automation platform has three primary interfaces that are validated through testing: the web-based intake form, the Monday.com integration board, and the confirmation email system powered by Microsoft Graph. The application is containerized with Docker and runs identically across all environments. Each component below is tested manually with screenshots in Section 2 and automatically with pytest in Section 3 where applicable.

i) User Authentication

- Valid intake login credentials authenticate the user and grant access to the intake forms
- Unauthenticated users attempting to access protected pages are redirected to the login page
- Invalid credentials are rejected and an error message is displayed
- Manual verification: Section 2 — i) Authentication
- Automated verification: Section 3 — iii) Authentication Tests (test_auth.py)

ii) Individual Intake Form Submission

- Valid individual form submission completes without errors
- Required fields are enforced — blank submissions are rejected with error messages
- Incorrect input formats (e.g., letters in a phone number field) display a validation error
- Manual verification: Section 2 — ii) Intake Form Submission
- Automated verification: Section 3 — iv) Form Submission Tests (test_forms.py)

iii) Business Intake Form Submission

- Valid business form submission completes without errors
- Required fields and input validation behave the same as the individual form
- Manual verification: Section 2 — ii) Intake Form Submission
- Automated verification: Section 3 — iv) Form Submission Tests (test_forms.py)

iv) Database Storage (PostgreSQL)

- Individual form submissions are written to the correct table in the PostgreSQL database
- Business form submissions are written to the correct table with accurate field values
- Manual verification: Section 2 — iii) Database Storage
- Automated verification: Section 3 — v) Database Tests (test_database.py)

v) PDF Generation System

- A PDF is generated after each successful intake form submission
- Form field data maps correctly to the corresponding PDF template fields

- Manual verification: Section 2 — iv) PDF Generation

vi) Monday.com Integration

- A new item is created on the Monday.com intake board after each successful submission
- Custom columns are populated correctly: Emails, Phone Numbers, Locations, Dates
- Manual verification: Section 2 — v) Monday.com Integration
- Automated verification: Section 3 — vi) Monday.com Tests (test_monday.py)

vii) Confirmation Email

- A confirmation email is sent to the submitter upon successful form completion
- Email content is accurate: correct sender, subject line, and body
- Manual verification: Section 2 — vi) Confirmation Email

viii) Docker Setup and Application Startup

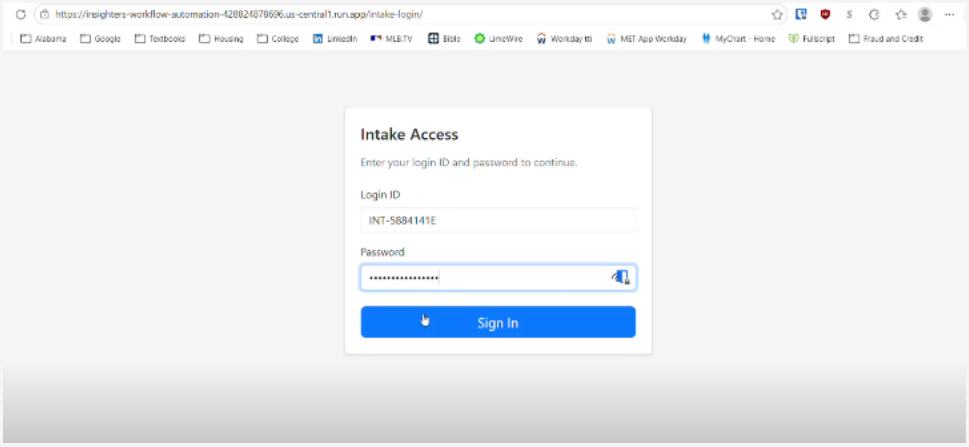
- docker compose up --build completes without errors
- The application starts and is accessible through the browser after startup
- Manual verification: Section 2 — vii) Docker Startup
- Automated verification: Section 3 — vii) Docker / Application Startup Tests (test_docker.py)

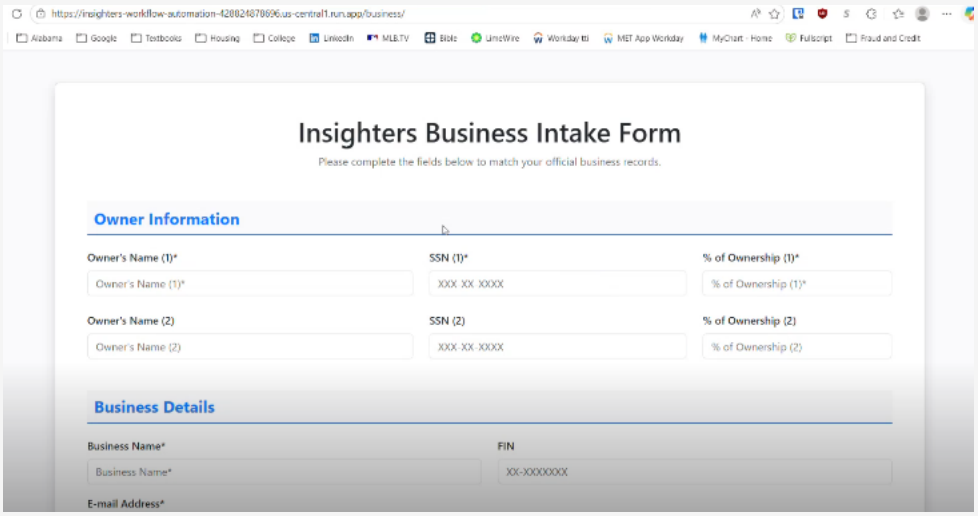
2. Significant Test Cases

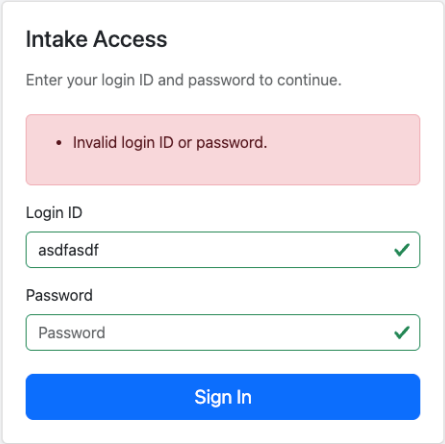
This section documents the manual workflow verification for each component of the application. Each subsection contains a table of test cases with a Pass/Fail column to be filled in during testing, followed by screenshot placeholders showing the expected visual evidence for each test.

i) Authentication

Test ID	Description	Expected Result	Pass/Fail
TC-AUTH-01	Valid intake credentials log the user in	User is authenticated and redirected to the intake form	pass
TC-AUTH-02	Accessing a protected page without logging in	User is redirected to the login page	pass
TC-AUTH-03	Invalid credentials submitted on the login page	Login fails; error message is shown	pass

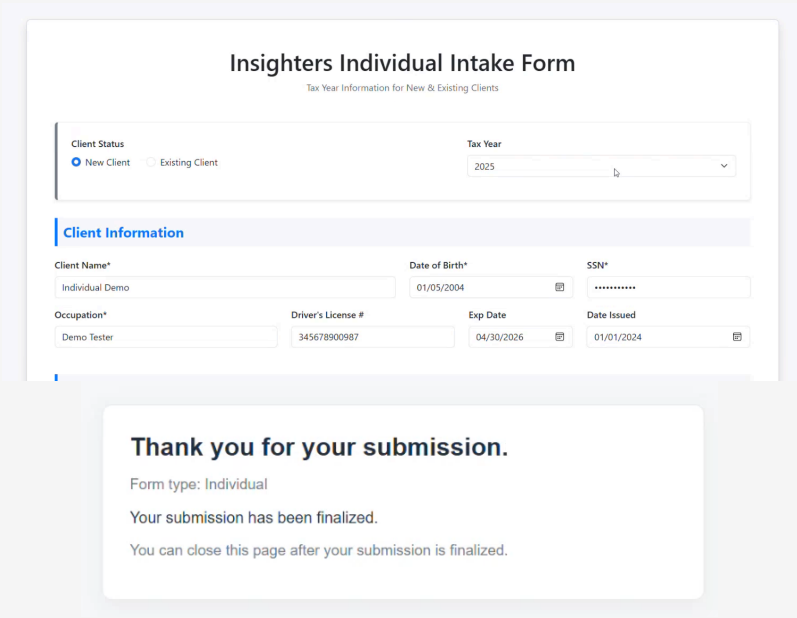


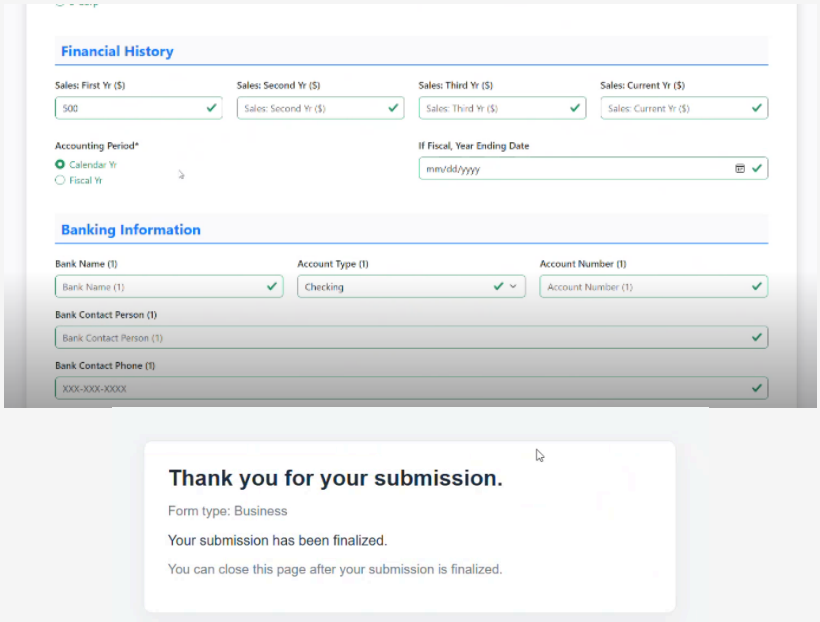

Insighters Business Intake Form
Please complete the fields below to match your official business records.
Owner Information
Owner's Name (1)*
Owner's Name (1)*
SSN (1)*
XXX-XX-XXXX
% of Ownership (1)*
% of Ownership (1)*
Owner's Name (2)
Owner's Name (2)
SSN (2)
XXX-XX-XXXX
% of Ownership (2)
% of Ownership (2)
Business Details
Business Name*
Business Name*
FIN
XX-XXXXXXX
E-mail Address*


Intake Access
Enter your login ID and password to continue.
Invalid login ID or password.
Login ID
asdfasdf
Password
Password
Sign In

ii) Intake Form Submission

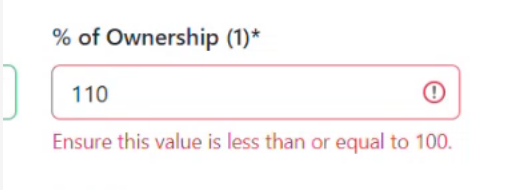
Test ID	Description	Expected Result	Pass/Fail
TC-FORM-01	Submit the individual intake form with all valid inputs	Form submits successfully	pass
TC-FORM-02	Submit the business intake form with all valid inputs	Form submits successfully	pass
TC-FORM-03	Submit a form with required fields left blank	Submission blocked; required field errors appear	pass
TC-FORM-04	Enter an invalid format in a field (e.g., letters in phone number)	Field validation rejects the input; error message shown	pass




Financial History
Sales: First Yr ($)
500
Sales: Second Yr ($)
Sales: Second Yr ($)
Sales: Third Yr ($)
Sales: Third Yr ($)
Sales: Current Yr ($)
Sales: Current Yr ($)
Accounting Period*
Calendar Yr
Fiscal Yr
If Fiscal, Year Ending Date
mm/dd/yyyy
Banking Information
Bank Name (1)
Bank Name (1)
Account Type (1)
Checking
Account Number (1)
Account Number (1)
Bank Contact Person (1)
Bank Contact Person (1)
Bank Contact Phone (1)
XXX-XXX-XXXX

Thank you for your submission.
Form type: Business
Your submission has been finalized.
You can close this page after your submission is finalized.

E-mail Address*
mail@mail.com
Address*
Address*
This field is required.


% of Ownership (1)*
110
Ensure this value is less than or equal to 100.

iii) Database Storage

Test ID	Description	Expected Result	Pass/Fail
TC-DB-01	Check the database after an individual form submission	A new record exists with the correct field values	pass
TC-DB-02	Check the database after a business form submission	A new record exists with the correct field values	pass

Results

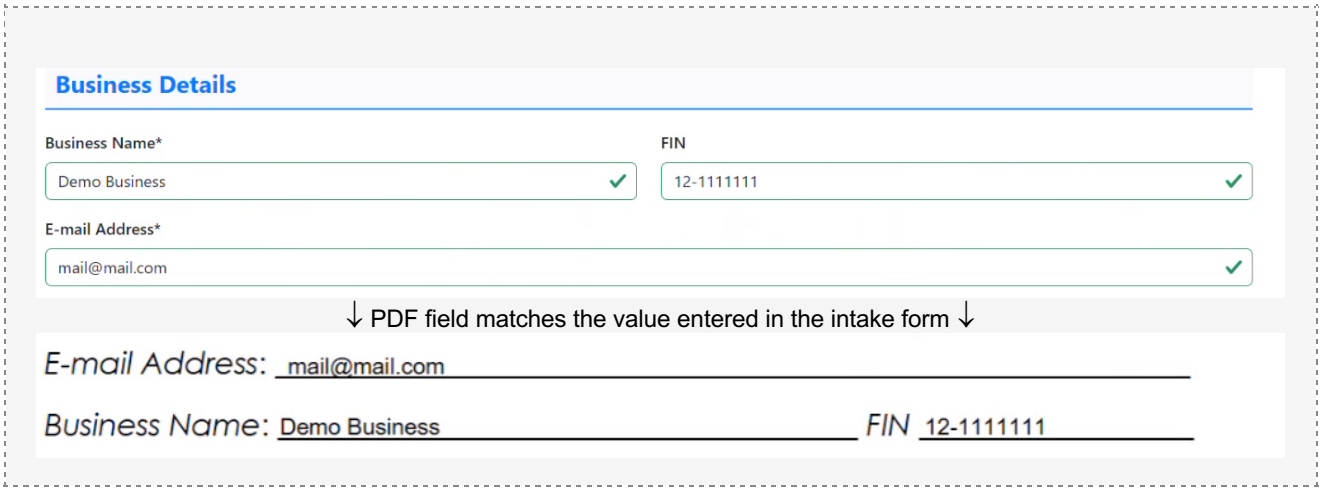
Execution time: 12.9 ms Export

id	created_at	client_status	tax_year	client_name	client_dob	client_occupation	client_dl	client_dl_exp	client_dl
1	2026-03-13T03:43:5...	new	2025	Cloud Test	2009-09-09...	Cloud Engineer			
2	2026-03-13T03:53:2...	new	2025	Cloud Test 2	2026-03-12...	Job 2			
3	2026-03-23T23:04:3...	new	2025	Alex DuBois	2001-01-01...	Printer			
4	2026-03-24T17:35:5...	new	2025	Keegan Test	2026-03-03...	Job			
5	2026-03-24T21:37:1...	new	2025	Ronald Mc...	2001-01-01...	Restauranteur			
6	2026-04-01T21:29:3...	new	2025	Security Te...	2026-04-01...	Security Consult...			
7	2026-04-09T19:35:0...	new	2025	John Doe	2026-04-09...	Unemployeed			
8	2026-04-09T20:21:5...	new	2025	Tim Cooke	2026-04-01...	Unemployee			
9	2026-04-14T14:53:3...	new	2025	Timezone ...	2008-01-01...	Tester			
10	2026-04-16T11:25:5...	new	2025	Secret Upd...	2026-04-16...	Individual			
11	2026-04-16T20:09:5...	new	2025	Individual ...	2004-01-05...	Demo Tester	345678900987	2026-04-30...	2024-01-

id	created_at	owner1_name	owner1_ownership	owner2_name	owner2_ownership	business_name	fin_number	email
1	2026-03-23T16:52:0...	New Test	50.00			Business		mail@mil.con
2	2026-04-09T19:31:1...	EJ	89.00			Sport Jones		s@j.com
3	2026-04-09T19:50:5...	Joe Momma	75.00			Joe's Shake S...		joe@momma
4	2026-04-09T20:30:2...	Timmy Tunner	54.00			Fairy Odd Foo...		f@o.com
5	2026-04-16T00:17:0...	Final Dev Test	100.00			Dev Business		mail@mail.cc
6	2026-04-16T11:28:0...	Business Se...	98.99			Business Secr...		mail@mail.cc
7	2026-04-16T20:02:3...	Project Dem...	100.00			Demo Business	12-11111...	mail@mail.cc
8	2026-04-16T20:38:1...	Prod Test	98.99			Production Te...		mail@mail.cc
9	2026-04-16T20:49:1...	Please Work	99.00	Must Work	0.50	Please	12-11111...	please@pleas
10	2026-04-16T20:52:2...	Please Work	98.00			yuhdufwdf		dfhhfb@fsjkd

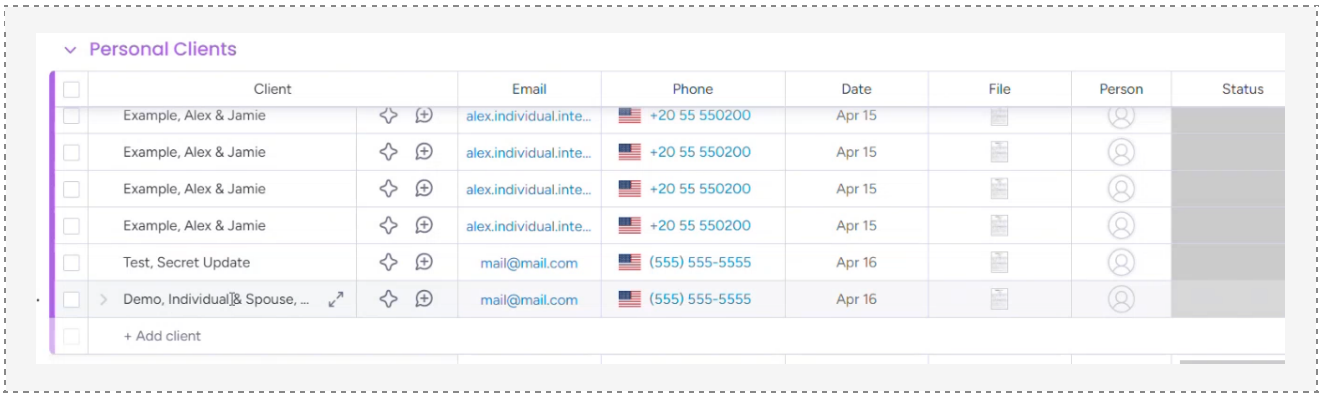
iv) PDF Generation

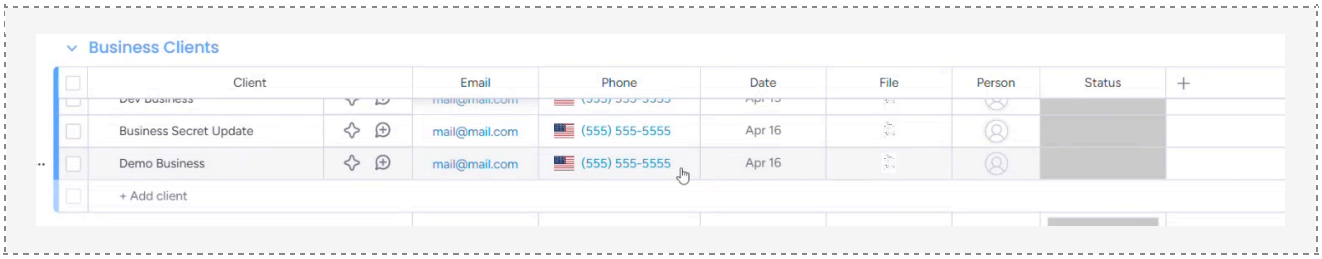
Test ID	Description	Expected Result	Pass/Fail
TC-PDF-01	A PDF is generated after a valid form submission	PDF file is created and opens successfully	pass
TC-PDF-02	Generated PDF contains the submitted form values	Each PDF field matches the value entered in the intake form	pass



v) Monday.com Integration

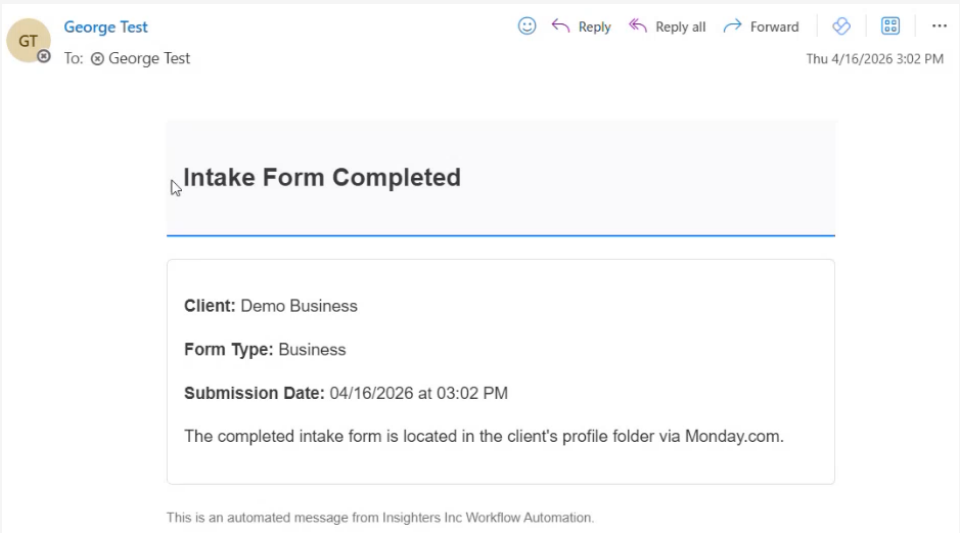
Test ID	Description	Expected Result	Pass/Fail
TC-MON-01	Submit a form and verify a new item appears on the Monday.com board	New item is created with the correct name	pass
TC-MON-02	Verify the item columns are populated correctly	Email, phone, location, and date columns contain the correct values	pass

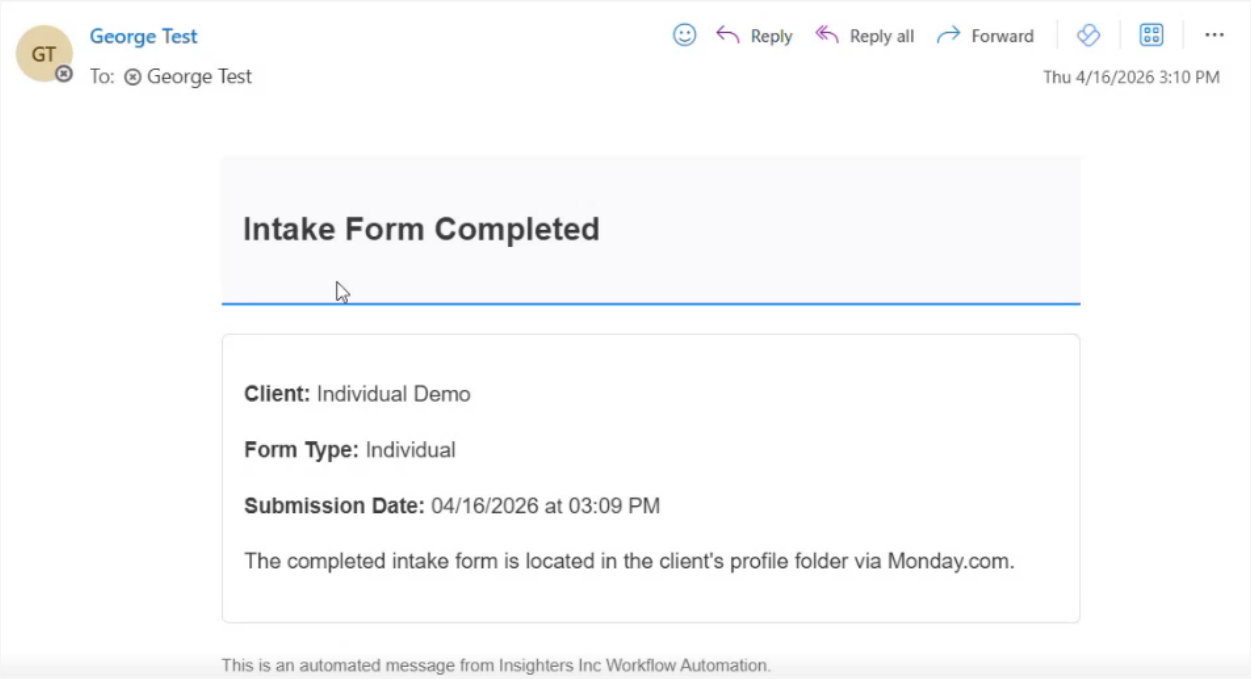




vi) Confirmation Email

Test ID	Description	Expected Result	Pass/Fail
TC-EMAIL-01	Submit a valid form and check for a confirmation email	Email is received with correct content	pass
TC-EMAIL-02	Verify the sender address, subject line, and body are correct	Email shows expected sender, subject, and body content	pass

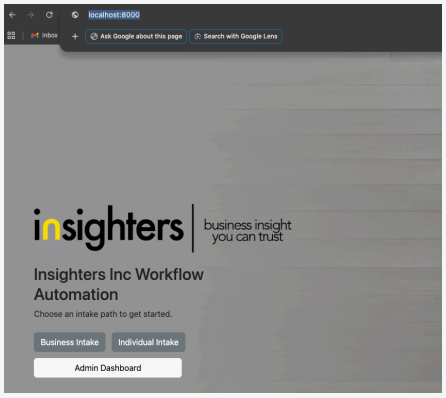




vii) Docker Startup

Test ID	Description	Expected Result	Pass/Fail
TC-DOCK-01	Run docker compose up --build from the project root	Build completes without errors; application starts successfully	pass
TC-DOCK-02	Verify the application is accessible in the browser after startup	Application home or login page loads correctly	pass

```
ayumacdc1@Ayushs-MacBook-Pro InsightersIncWorkflowAutomation % docker compose up -d --build
[+] Building 0.4s (16/16) FINISHED
 => [internal] load local bake definitions                                                    0.0s
 => => reading from stdin 612B                                                                0.0s
 => [internal] load build definition from Dockerfile                                          0.0s
 => => transferring dockerfile: 1.67kB                                                        0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                           0.2s
 => [internal] load .dockerignore                                                             0.0s
 => => transferring context: 315B                                                             0.0s
 => [1/9] FROM docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78  0.0s
 => => resolve docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78  0.0s
 => [internal] load build context                                                             0.0s
 => => transferring context: 4.18kB                                                           0.0s
 => CACHED [2/9] RUN addgroup --system appgroup && adduser --system --ingroup appgroup appuser  0.0s
 => CACHED [3/9] WORKDIR /app                                                                 0.0s
 => CACHED [4/9] RUN apt-get update && apt-get install -y     libpq-dev     gcc     && rm -rf /  0.0s
 => CACHED [5/9] COPY requirements.txt /app/                                                  0.0s
 => CACHED [6/9] RUN pip install --no-cache-dir -r requirements.txt                           0.0s
 => CACHED [7/9] COPY --chown=appuser:appgroup . /app/                                        0.0s
 => CACHED [8/9] RUN chown appuser:appgroup /app && mkdir -p /app/staticfiles && chown appuser:  0.0s
 => CACHED [9/9] RUN sed -i 's/\r$//' /app/scripts/entrypoint.sh && chmod +x /app/scripts/entry  0.0s
 => exporting to image                                                                        0.0s
 => => exporting layers                                                                       0.0s
 => => exporting manifest sha256:e3fe7144a883d57c36a7870d3f8cbae82b18a9f2ebc39938103e6c9090586b  0.0s
 => => exporting config sha256:ad85f830a32c0fcbccb84b4c07e8a85f16f99ee2a6c3c86137c7dd93de55235e  0.0s
 => => exporting attestation manifest sha256:75be790e67281cb84bbf28b9a1ea7ef203294f853beb23e662  0.0s
 => => exporting manifest list sha256:f3da99f6a57c5a5ac82e639d34d27cb9e1ffae73230cc9fffdb96e351  0.0s
 => => naming to docker.io/library/insightersincworkflowautomation-web:latest                 0.0s
 => => unpacking to docker.io/library/insightersincworkflowautomation-web:latest              0.0s
 => resolving provenance for metadata file                                                    0.0s
[+] up 4/4
 ✔ Image insightersincworkflowautomation-web       Built                                      0.5s
 ✔ Network insightersincworkflowautomation_default Created                                    0.0s
 ✔ Container insightersincworkflowautomation-db-1  Healthy                                    5.7s
 ✔ Container insightersincworkflowautomation-web-1 Created                                    0.0s
ayumacdc1@Ayushs-MacBook-Pro InsightersIncWorkflowAutomation %
```



3. Test Scripts

The following test scripts are written in Python using the pytest framework. The suite is organized into six test and support files, each targeting a specific component of the application, along with a shared __init__.py. All scripts live in a /tests folder at the root of the project and share a common conftest.py that handles environment setup, authentication helpers, and test data fixtures.

The suite reads configuration directly from the project .env file, so no hardcoded credentials or URLs appear in any test file. BASE_URL defaults to http://localhost:8000 and can be changed to point at another environment including Cloud Run. Note that these are integration tests — some of them submit real form data to the running application, which can create database records and generated PDFs, so they are intentionally run against the development environment rather than production.

i) Setup and Running Tests

Install the required testing dependencies:

```
python3 -m pip install pytest requests psycopg2-binary
```

Start the application, then run the full suite from the project root:

```
docker compose up --build

python3 -m pytest tests/ -v
```

```
● ayumacdc1@Ayushs-MacBook-Pro InsightersIncWorkflowAutomation % python3 -m pytest tests/ -v

  ========================================================== test session starts ==========================================================
  platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0 -- /Library/Developer/CommandLineTools/usr/bin/python3
  cachedir: .pytest_cache
  rootdir: /Users/ayumacdc1/Documents/oth/InsightersIncWorkflowAutomation
  collected 12 items

  tests/test_auth.py::test_dashboard_redirects_when_not_authenticated PASSED                                                    [  8%]
  tests/test_auth.py::test_intake_login_allows_env_credentials_for_individual_form PASSED                                       [ 16%]
  tests/test_auth.py::test_intake_login_rejects_bad_credentials PASSED                                                          [ 25%]
  tests/test_database.py::test_business_submission_persists_and_does_not_echo_sensitive_values PASSED                           [ 33%]
  tests/test_database.py::test_individual_submission_persists_and_does_not_echo_sensitive_values PASSED                         [ 41%]
  tests/test_docker.py::test_app_root_is_reachable PASSED                                                                        [ 50%]
  tests/test_docker.py::test_login_pages_load_when_container_is_running PASSED                                                   [ 58%]
  tests/test_forms.py::test_individual_form_accepts_valid_fake_submission PASSED                                                 [ 66%]
  tests/test_forms.py::test_business_form_accepts_valid_fake_submission PASSED                                                   [ 75%]
  tests/test_forms.py::test_empty_individual_form_is_rejected PASSED                                                             [ 83%]
  tests/test_forms.py::test_empty_business_form_is_rejected PASSED                                                               [ 91%]
  tests/test_monday.py::test_dev_monday_token_and_board_are_reachable PASSED                                                     [100%]

  ============================================================ warnings summary ============================================================
  ../../../Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35
    /Users/ayumacdc1/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL
  1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
      warnings.warn(

  -- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
  ================================================== 12 passed, 1 warning in 12.60s ==================================================
○ ayumacdc1@Ayushs-MacBook-Pro InsightersIncWorkflowAutomation %
```

ii) conftest.py — Shared Fixtures and Helpers

conftest.py is the backbone of the test suite. It loads the .env file automatically so every test has access to credentials and configuration without importing anything manually. It provides three pytest fixtures used across all test files:

- base_url — reads BASE_URL from the environment, defaults to http://localhost:8000
- session — creates a fresh requests.Session() for each test and closes it when done
- valid_individual_data and valid_business_data — fully populated fake payloads with realistic Alabama-based data used to simulate real form submissions

It also provides three helper functions that every test file imports:

- build_url(base_url, path) — safely joins the base URL and a path
- post_with_csrf(session, base_url, path, data) — fetches the form page first to extract the Django CSRF token, then posts the form data with the correct headers. This is required because Django rejects any POST without a valid CSRF token.
- login_for_intake_path(session, base_url, path) — reads INTAKE_LOGIN_ID and INTAKE_LOGIN_PASSWORD from .env, navigates to a protected path to trigger the login redirect, then submits the intake login form with the correct CSRF token

```
# conftest.py — key sections

def login_for_intake_path(session, base_url, protected_path):
    login_id = os.environ.get('INTAKE_LOGIN_ID')
    password = os.environ.get('INTAKE_LOGIN_PASSWORD')
    # Navigates to the protected path first to trigger the redirect,
    # then posts credentials to /intake-login/ with a valid CSRF token
    preflight = session.get(build_url(base_url, protected_path),
                            allow_redirects=False, timeout=30)
    return post_with_csrf(session, base_url, '/intake-login/',
                          {'login_id': login_id, 'password': password},
                          allow_redirects=False, timeout=30)

def post_with_csrf(session, base_url, path, data, **kwargs):
    # Fetches the form page to extract the csrfmiddlewaretoken,
    # then injects it into the POST payload automatically
    form_page = session.get(build_url(base_url, path), timeout=30)
    csrf = re.search(r'name="csrfmiddlewaretoken" value="([^"]+)"',
                     form_page.text).group(1)
    return session.post(build_url(base_url, path),
                        data={'csrfmiddlewaretoken': csrf, **data},
                        headers={'Referer': build_url(base_url, path)},
                        **kwargs)
```

iii) Authentication Tests — test_auth.py

Three tests covering the full authentication surface of the application:

- test_dashboard_redirects_when_not_authenticated — makes an unauthenticated GET request to /dashboard/ and confirms the response is a redirect to /admin-login/
- test_intake_login_allows_env_credentials_for_individual_form — uses the login helper to authenticate with the .env credentials and confirms the redirect lands on /individual/
- test_intake_login_rejects_bad_credentials — posts deliberately wrong credentials to /intake-login/ and confirms the response contains the expected error message

```
# test_auth.py

def test_dashboard_redirects_when_not_authenticated(session, base_url):
    response = session.get(build_url(base_url, '/dashboard/'),
                           allow_redirects=False, timeout=30)
    assert response.status_code in {302, 303}
    assert '/admin-login/' in response.headers.get('Location', '')

def test_intake_login_allows_env_credentials_for_individual_form(session,
base_url):
    login_response = login_for_intake_path(session, base_url, '/individual/')
    assert login_response.status_code in {302, 303}
    assert '/individual/' in login_response.headers.get('Location', '')

def test_intake_login_rejects_bad_credentials(session, base_url):
    # First triggers the redirect to confirm the route is protected
    session.get(build_url(base_url, '/individual/'),
                allow_redirects=False, timeout=30)
    response = post_with_csrf(session, base_url, '/intake-login/',
                              {'login_id': 'bad-user', 'password': 'bad-password'},
                              timeout=30)
    assert response.status_code == 200
    assert 'Invalid login ID or password' in response.text
```

iv) Form Submission Tests — test_forms.py

Four tests covering valid and invalid submissions for both intake forms:

- test_individual_form_accepts_valid_fake_submission — authenticates, then posts a fully populated individual payload to /individual/ and confirms no validation error messages appear in the response
- test_business_form_accepts_valid_fake_submission — same flow for the business form at /business/
- test_empty_individual_form_is_rejected — posts an empty payload to /individual/ and confirms the response contains a required field error
- test_empty_business_form_is_rejected — same for the business form

```
# test_forms.py

def test_individual_form_accepts_valid_fake_submission(
        session, base_url, valid_individual_data):
    login_response = login_for_intake_path(session, base_url, '/individual/')
    assert login_response.status_code in {302, 303}
    response = post_with_csrf(session, base_url, '/individual/',
                              valid_individual_data,
                              allow_redirects=True, timeout=60)
    assert response.status_code == 200
    assert 'This field is required' not in response.text
    assert 'Personal Form Validation Errors' not in response.text

def test_empty_individual_form_is_rejected(session, base_url):
    login_response = login_for_intake_path(session, base_url, '/individual/')
    assert login_response.status_code in {302, 303}
    response = post_with_csrf(session, base_url, '/individual/', {},
                              timeout=30)
    assert response.status_code == 200
    assert 'This field is required' in response.text
```

v) Database Tests — test_database.py

Two tests that verify form submissions are persisted correctly in PostgreSQL and that sensitive values are never exposed in HTTP responses:

- test_individual_submission_persists_and_does_not_echo_sensitive_values — submits the individual form, confirms the HTTP response does not contain the test SSN or spouse SSN, then connects directly to the database and queries core_personalintakesubmission to confirm the record exists with the correct email, name, and filing status
- test_business_submission_persists_and_does_not_echo_sensitive_values — same pattern for the business form, checking core_businessintakesubmission for the correct email, bank account type, and business name, and confirming SSNs and bank account numbers are absent from the HTTP response

The database connection is built by parsing the DATABASE_URL environment variable directly from .env, so no credentials are hardcoded. If DATABASE_URL is not set, the tests skip automatically.

```
# test_database.py — key section

def test_individual_submission_persists_and_does_not_echo_sensitive_values(
        session, base_url, valid_individual_data):
    login_for_intake_path(session, base_url, '/individual/')
    response = post_with_csrf(session, base_url, '/individual/',
                              valid_individual_data,
                              allow_redirects=True, timeout=60)
    # Confirm sensitive values are not in the HTTP response
    assert valid_individual_data['client_ssn'] not in response.text
    assert valid_individual_data['spouse_ssn'] not in response.text

    # Connect directly to PostgreSQL and verify the record was saved
    conn = psycopg2.connect(**parsed_database_url_from_env)
    cursor.execute(
        'SELECT email, client_name, filing_status'
        ' FROM core_personalintakesubmission'
        ' WHERE email = %s ORDER BY created_at DESC LIMIT 1',
        (valid_individual_data['email'],))
    row = cursor.fetchone()
    assert row is not None
    assert row[0] == valid_individual_data['email']
```

vi) Monday.com Tests — test_monday.py

One test that directly pings the Monday.com API to confirm the integration is configured and reachable:

- test_dev_monday_token_and_board_are_reachable — reads DEV_MONDAY_API and DEV_MONDAY_BOARD_ID from .env, posts a GraphQL query to the Monday.com v2 API, and confirms the response is successful, contains no errors, and returns the expected board ID. If the environment variables are not set the test skips automatically.

```
# test_monday.py

def test_dev_monday_token_and_board_are_reachable():
    token    = os.environ.get('DEV_MONDAY_API')
    board_id = os.environ.get('DEV_MONDAY_BOARD_ID')
    if not token or not board_id:
        pytest.skip('DEV_MONDAY_API and DEV_MONDAY_BOARD_ID required.')

    response = requests.post(
        'https://api.monday.com/v2',
        headers={'Authorization': token,
                 'Content-Type': 'application/json',
                 'API-Version': '2024-04'},
        json={'query': 'query ($board_id: [ID!])'
                       '{ boards(ids: $board_id) { id name } }',
              'variables': {'board_id': [str(board_id)]}},
        timeout=30,
    )
    assert response.status_code == 200
    payload = response.json()
    assert 'errors' not in payload
    assert str(payload['data']['boards'][0]['id']) == str(board_id)
```

vii) Docker / Application Startup Tests — test_docker.py

Two smoke tests that confirm the containerized application is running and serving pages correctly:

- test_app_root_is_reachable — makes a GET request to the application root and asserts a 200 response, confirming the container started and the web server is accepting connections
- test_login_pages_load_when_container_is_running — fetches both /intake-login/ and /admin-login/ and confirms both return 200 and contain the expected page content, confirming the application is serving expected pages after startup

```
# test_docker.py

def test_app_root_is_reachable(session, base_url):
    response = session.get(build_url(base_url, '/'), timeout=30)
    assert response.status_code == 200

def test_login_pages_load_when_container_is_running(session, base_url):
    intake_login = session.get(build_url(base_url, '/intake-login/'), timeout=30)
    admin_login  = session.get(build_url(base_url, '/admin-login/'),  timeout=30)
    assert intake_login.status_code == 200
    assert admin_login.status_code  == 200
    assert 'Login ID'     in intake_login.text
    assert 'Admin Access' in admin_login.text
```

> *All 12 tests passed successfully against the running application. See the pytest terminal output screenshot at the top of this section for the full results.*